

Práctica 3: Búsqueda con Adversarios (Minimax)

1. Introducción

El objetivo de esta práctica es crear un juego en el cual dos jugadores se mueven por un laberinto dejando un rastro según van pasando por las casillas. El rastro se convierte en pared, es decir, ya no pueden volver a pasar por donde ya pasaron. Un jugador pierde si choca con una pared, con el rastro del rival o con su propio rastro, o si directamente se queda sin movimientos disponibles. El jugador A empieza en la columna izquierda y su objetivo es llegar a la columna derecha, y el jugador B a la inversa. También pueden ganar si el rival se queda sin movimientos.

Para esta práctica hacemos uso de la base adquirida de las prácticas anteriores, en las cuales el agente estaba solo en el laberinto y se implementaron algoritmos de búsqueda para encontrar el camino más corto desde la entrada a la salida. También he hecho uso del código base proporcionado, y del pseudocódigo de las diapositivas de la asignatura.

2. Diseño de los Algoritmos

2.1 Arquitectura del código

Empezamos con la base proporcionada: `Tron_P3_Base.java`, que incluye la clase `Entorno` que gestiona el tablero, detecta colisiones y alterna turnos, y la clase `AgenteDummy` que elige un movimiento aleatorio entre los válidos. A partir de aquí implementé las clases `BusquedaMiniMax` y `BusquedaAlfaBeta`.

Los métodos más usados y por tanto más importantes para esta práctica han sido `moverAgente` y `deshacerMovimiento`, ya que permiten simular movimientos en el tablero sin tener que hacer copias del tablero. Para implementar ambos algoritmos que realizado un movimiento, calculado el valor de ese futuro haciendo una llamada recursiva al algoritmo y luego he deshecho el movimiento para dejar el tablero como estaba. De esta forma el agente puede explorar el árbol de juego.

Para evitar que el código fuese muy repetitivo, he extraído las funciones de evaluación a una clase separada llamada “Heurística” con dos métodos: “`utilidad`” y “`utilidadSimple`” que implementan la heurística completa y solo Manhattan respectivamente.

2.2 Minimax con profundidad limitada

El árbol de juego es demasiado grande para explorarlo por completo, así que se usa una profundidad máxima. En Minimax he puesto 4 y en AlfaBeta 6 porque la poda permite llegar más lejos en el mismo tiempo.

He seguido exactamente el pseudocódigo de las diapositivas del tema 5 de la asignatura, implementando tres funciones separadas:

- DECISIÓN-MINIMAX (pensar): Prueba las 4 acciones posibles para el jugador A, simula cada una, llama a minValor para que B explore su respuesta, deshace el movimiento y se queda con la acción que dio mayor valor. Devuelve una acción, no un número.
- MAX-VALOR (maxValor): representa el turno de A dentro del árbol. A quiere maximizar su puntuación. Empieza con $v = -\text{INFINITO}$ y para cada movimiento válido llama a minValor (turno de B) y se queda con el valor más alto. Si se llega a profundidad 0 o el juego ha terminado, llama a utilidad en vez de seguir bajando.
- MIN-VALOR (minValor): representa el turno de B dentro del árbol. B quiere minimizar la puntuación de A para perjudicarlo. Empieza con $v = +\text{INFINITO}$ y se queda con el valor más bajo que encuentre.

La condición de parada (TEST-TERMINAL) se cumple si hemos llegado al límite configurado, A no tiene movimientos y pierde o si B no tiene movimientos y por tanto A gana.

2.3 Función de Evaluación Heurística

Cuando el algoritmo llega a un nodo con profundidad 0, estima lo bueno que es ese estado para A sin saber exactamente cómo va a terminar la partida. Para eso uso la función utilidad.

Primero compruebo si A o B han terminado, es decir, si no tienen movimientos. Si B no tiene devuelvo $+\text{INFINITO}$ por lo que A ha ganado, y en caso contrario devuelvo $-\text{INFINITO}$ por lo que A ha perdido. Para el resto de estados (los estados intermedios) he combinado dos heurísticas: diferencia de movilidad y distancia Manhattan a la meta. Le he dado más importancia a la movilidad (1 movimiento libre valdrá 10 puntos) que a la distancia (2 puntos) porque en Tron sobrevivir es más importante que avanzar, así que hago $\text{utilidad} = \text{difMovilidad} * 10 - \text{distA} * 2$ siendo $\text{difMovilidad} = \text{movimientosValidos('A')} - \text{movimientosValidos('B')}$ (la primera heurística y $\text{distA} = |\text{columna_actual_A} - \text{columna_meta_A}|$ la segunda.

Como se pedía en el enunciado, he implementado otra heurística más simple que solo tiene en cuenta la distancia Manhattan: $\text{utilidad} = -\text{distA} * 2$. Esta heurística hace que el agente avance a la meta sin tener en cuenta si tiene movimientos libres. Sin embargo, para las pruebas se usará la

heurística más completa. Como prueba de funcionamiento, ejecutando Minimax con profundidad 4 vs AgenteDummy, el agente gana en 5 ciclos avanzando hacia el Este mientras mantiene más movimientos libres que el rival.

2.4 Poda Alfa-Beta

Es el mismo algoritmo que el implementado para min-max pero con dos parámetros extra que se pasan de padres a hijos:

- Alfa: empieza en -INFINITO porque es el mejor valor que max tiene en algún otro camino explorado
- Beta: empieza en +INFINITO porque es el mejor valor que min ya tiene garantizado

Esta poda lo que hace es que si estamos un nodo MAX, si el mejor valor encontrado hasta ahora es mayor o igual que beta, MIN nunca elegiría ese nodo porque ya tiene un camino mejor en otro lugar del árbol, así que cortamos el bucle sin explorar el resto de hijos y devolvemos el mejor valor encontrado. En un nodo MIN, si el mejor valor encontrado hasta ahora es menor o igual que alfa, MAX nunca elegiría ese nodo porque ya tiene un camino mejor, así que cortamos y devolvemos el mejor valor encontrado.

3. Análisis de profundidad

Ejecutando minmax a distintas profundidades (usando mapaTexto) que observado este comportamiento:

PROFUNDIDAD	NODOS VISITADOS	CICLOS HASTA GANAR
2	4 a 12	10
4	2 a 92	23
6	2 a 614	10

Viendo estos datos podemos concluir que a profundidad 2, como el agente solo ve 2 movimientos hacia delante, toma decisiones cortas de vista. Sin embargo a profundidad 4 este comportamiento mejora aunque tarde más ciclos en ganar y a 6 aumenta mucho el número de nodos pero el agente juega de forma más inteligente viendo más turnos adelante. Respecto a la memoria, es proporcional a la profundidad máxima ya que va guardando el camino actual, así que a profundidad 2 tiene máximo 2 llamadas anidadas, a 4 tiene 4 y a 6 tiene 6.

Y ejecutando AlfaBeta de esta misma forma observamos el siguiente:

PROFUNDIDAD	NODOS VISITADOS	CICLOS HASTA GANAR
2	8 a 12	13
4	2 a 73	13
6	2 a 222	15

AlfaBeta es similar a Minimax en cuanto a ciclos pero con menos nodos visitados gracias a la poda. El consumo de memoria es igual a Minimax ya que la poda solo afecta al número de ramas exploradas.

4. Comparativa AlfaBeta vs Minimax

Enfrentando a estos algoritmos obtenemos:

PROFUNDIDAD	NODOS MINIMAX	NODOS ALFABETA	CICLOS HASTA GANAR
2	3/9	4/12	13
4	0/55	1/59	19
6	0/482	2/319	7

Podemos observar que con la misma heurística y profundidad, ambos dan el mismo resultado, pero AlfaBeta explora menos nodos gracias a la poda. Esta reducción es mayor cuanto más profundidad se usa, porque según avanzas hay más ramas que pueden ser podadas. Es decir que a profundidades bajas la ventaja no es visible. Sin embargo a profundidad 4 AlfaBeta visita más nodos que Minimax porque al enfrentarse a otro agente el árbol es más simétrico y hay menos ramas que podar.

Sin embargo, estos dos algoritmos contra el AgenteDummy dan los siguientes resultados:

PROFUNDIDAD	NODOS MINIMAX	NODOS ALFABETA	REDUCCIÓN MÁX (%)
2	4/12	4/12	0
4	2/92	2/73	21
6	2/614	2/222	64

A profundidad 2 no se observa diferencia, pero a 4 AlfaBeta hace una mejora del 21% frente a Minimax, y a 6 una mejora del 64%, lo que confirma que AlfaBeta es más efectiva cuanto mayor es la profundidad, como se vio en clase de teoría.

5. Simulaciones

5.1 Minimax (profundidad 4) vs AgenteDummy

```
--- Ciclo 0 ---
#####
#A          #
#           #
#           #
#           #
#  #       B #
#####
Nodos visitados MinMax: 44
[Agente A] Decide mover: S
[Agente B] Decide mover: N

--- Ciclo 3 ---
#####
#a          #
#aaaA       #
#           #
#           Bbb #
#  #       b #
#####
Nodos visitados MinMax: 92
[Agente A] Decide mover: E
[Agente B] Decide mover: S

--- Ciclo 5 ---
#####
#a          #
#aaaaA      #
#           #
#           bbb #
#  #       bBb #
#####
Nodos visitados MinMax: 3
[Agente A] Decide mover: N
[Agente B] Se ha quedado sin movimientos útiles a la vista. GANA EL JUGADOR A.
```

Con estos tres ciclos vemos que minimax ganó en 5 ciclos y que en el ciclo 0 ambos parten de sus posiciones iniciales. En el ciclo 3 vemos como minimax avanza en línea recta hacia el Este mientras que dummy va al sur de forma que se encierra en la esquina inferior derecha. En el ciclo 5 el dummy se queda sin movimientos válidos sin que minimax lo haya bloqueado, simplemente se destruyó solo.

5.2 Alfa-Beta (profundidad 6) vs Minimax(profundidad 4)

```
--- Ciclo 0 ---
#####
#A          #
#           #
#           #
#           #
# #         B #
#####
Nodos visitados AlfaBeta: 114
[Agente A] Decide mover: S
Nodos visitados MinMax: 51
[Agente B] Decide mover: E

--- Ciclo 10 ---
#####
#a          bb#
#aaaaaaa   bb#
#         a   bb#
#        aA   Bbb#
# #         bb#
#####
Nodos visitados AlfaBeta: 132
[Agente A] Decide mover: N
Nodos visitados MinMax: 51
[Agente B] Decide mover: S

--- Ciclo 17 ---
#####
#a          aa bb#
#aaaaaaa   aA bb#
#         aaaa bb#
#        aabbBbbb#
# #        bbbbbb#
#####
Nodos visitados AlfaBeta: 5
[Agente A] Decide mover: S
Nodos visitados MinMax: 0
[Agente B] Se ha quedado sin movimientos útiles a la vista. GANA EL JUGADOR A.
```

Vemos que alfabeta gana en 17 ciclos y en el ciclo 0 ambos parten iguales. En el ciclo 10 se aprecia la diferencia de profundidad porque AlfaBeta toma una ruta más larga pero más segura hacia el sur mientras que minimax avanza más directo sin ver las consecuencias a largo plazo. En el ciclo 17 minimax se ha quedado sin movimientos porque alfabeta ha conseguido acorralarlo sin que minimax lo detectara.

6. Conclusiones

La conclusión más relevante de esta práctica es que la heurística es la parte más importante de un algoritmo, ya que determina lo que el agente determina bueno. Si la heurística fuera mala, el agente tomaría decisiones incorrectas. En mi caso, he priorizado la movilidad sobre la distancia para que el agente no se quede encerrado con el objetivo de avanzar más rápido, lo que resulta en un comportamiento más inteligente en Tron.

La poda Alfa-Beta no cambia el resultado de Minimax, pero permite explorar mucha más profundidad aproximadamente en el mismo tiempo, y de esta forma podemos decir que el agente juega mejor.